

Rapport du Projet Moteur 3D

Diarra Amadou

Année universitaire 2024–2025

1 Introduction

Dans le cadre du projet de Moteur 3D, l'objectif était de mettre en œuvre un ou plusieurs effets de rendu temps-réel étudiés en cours et en travaux pratiques. J'ai choisi d'implémenter deux effets visuels : le **FXAA (Fast Approximate Anti-Aliasing)** et le **shadow mapping**. Le rendu a été réalisé à partir d'un modèle 3D complexe représentant une scène de style Minecraft.

2 Présentation de la scène

Le modèle utilisé est `vokselia.spawn.obj`, téléchargé depuis le site <https://www.planetminecraft.com/project/vokselia-spawn/>, qui propose des modèles Minecraft libres de droits. La scène est composée de 99 meshes, représentant plus de 1,8 million de triangles.

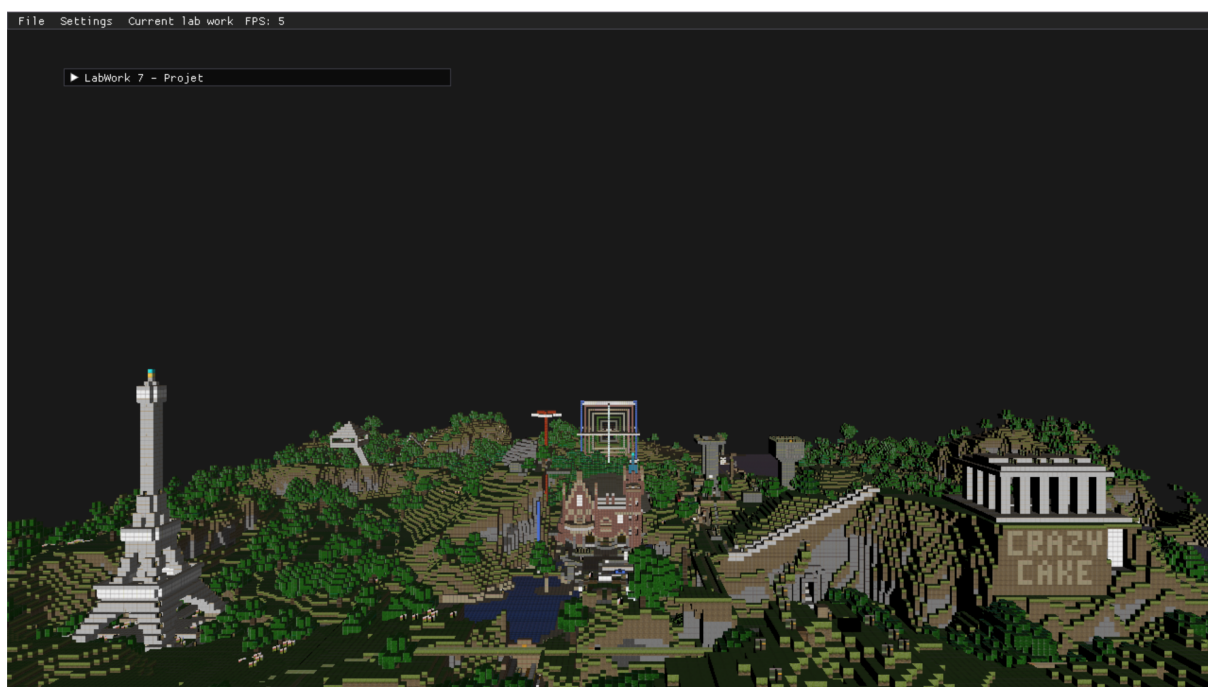


FIGURE 1 – Vue d'ensemble de la scène importée

Pour enrichir le rendu, j'ai activé différentes textures associées aux matériaux (diffuse, normale, spéculaire), générées via l'outil <https://cpetry.github.io/NormalMap-Online/>.

3 Effets implémentés

3.1 Effet visuel : FXAA (Fast Approximate Anti-Aliasing)

3.1.1 Présentation

FXAA (Fast Approximate Anti-Aliasing) est un algorithme de post-traitement conçu pour réduire l'aliasing, c'est-à-dire les effets de crénelage (bords en escalier) visibles sur les diagonales ou les contours fins des objets 3D. Contrairement au MSAA, FXAA est indépendant du processus de rasterisation : il s'applique à l'image finale, après le rendu, ce qui le rend très peu coûteux en ressources.

Dans ce projet, FXAA est utilisé pour améliorer la lisibilité de la scène, en particulier dans les zones contenant des détails géométriques fins. Il peut être activé ou désactivé dynamiquement via l'interface utilisateur (ImGui).

3.1.2 Détail de l'implémentation

FXAA fonctionne selon le principe suivant :

1. **Détection de contour** : le shader évalue la luminance du pixel courant et de ses voisins.
2. **Détection de contraste** : si un contraste fort est détecté, l'algorithme déduit qu'il s'agit d'un bord.
3. **Flou directionnel** : le shader applique un léger flou perpendiculaire au bord détecté, sur un rayon de quelques pixels.

Ce traitement est effectué dans un shader fragment appliqué sur un *fullscreen quad*, avec la texture couleur de la scène en entrée.

L'appel de cette passe est réalisé ainsi :

Listing 1 – Passe FXAA en C++

```
void LabWork7::_fxaaPass()
{
    if (!_useFXAA) return;
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    glUseProgram(_fxaaProgram);
    glBindTextureUnit(0, _sceneColorTex);
    glBindVertexArray(_quadVAO);
    glDrawArrays(GL_TRIANGLE_STRIP, 0, 4);
}
```

Le shader FXAA utilisé est inspiré d'un exemple open-source :

<https://github.com/mitsuhiko/webgl-meincraft/blob/master/assets/shaders/fxaa.glsl>

Les coordonnées UV sont interpolées automatiquement par le quad plein écran, et le shader utilise un accès local à ses voisins pour décider du traitement anti-crénélage.

3.1.3 Justification des choix

FXAA a été choisi pour plusieurs raisons :

- **Simplicité d'implémentation** : il ne nécessite qu'un shader fragment simple et une texture d'entrée, sans modification du pipeline de rendu.
- **Performances excellentes** : contrairement au MSAA ou TAA, FXAA ne dépend pas de la résolution du framebuffer ni du nombre d'échantillons. Il est donc adapté aux scènes complexes comme celle utilisée dans ce projet (plus de 1,8 million de triangles).
- **Qualité visuelle suffisante** : bien qu'il ne soit pas aussi précis que les techniques multi-échantillons, FXAA élimine efficacement l'aliasing sur les bords fins, notamment en arrière-plan.

- **Contrôle dynamique via UI** : FXAA peut être activé ou désactivé via ImGui, ce qui permet de tester son effet visuel facilement.

Ce choix est donc un bon compromis entre performance, lisibilité et simplicité de mise en œuvre.

3.2 Effet visuel : Shadow Mapping

3.2.1 Présentation

Le **shadow mapping** est une technique classique de rendu temps réel permettant de générer des ombres projetées dynamiques. Elle consiste à effectuer un rendu de la scène depuis le point de vue de la lumière, en capturant uniquement la profondeur des objets dans une texture appelée **shadow map**.

Lors du rendu final, chaque fragment est transformé dans l'espace de la lumière, et comparé à la profondeur enregistrée dans cette shadow map. Si le fragment est plus éloigné que ce que la lumière a vu, il est considéré comme étant dans l'ombre.

Cette technique fonctionne quelle que soit la complexité géométrique de la scène, et convient bien à des lumières directionnelles comme celle utilisée dans ce projet.

3.2.2 Détail de l'implémentation

Le shadow mapping a été implémenté en deux passes distinctes :

1. **Shadow Pass** : rendu de la scène depuis le point de vue de la lumière, avec un shader n'écrivant que la profondeur. La matrice `lightViewProj` est calculée à l'aide de `glm::lookAt` et `glm::ortho`.

Listing 2 – Extrait du shadow pass

```
glBindFramebuffer(GL_FRAMEBUFFER, _shadowFBO);
glViewport(0, 0, SHADOW_RES, SHADOW_RES);
glClear(GL_DEPTH_BUFFER_BIT);

Mat4f lightView = glm::lookAt(_lightPosition, Vec3f(0.f), Vec3f(0.f, 1.f,
    0.f));
Mat4f lightProj = glm::ortho(-s, s, -s, s, 0.1f, 30.f);
_lightViewProj = lightProj * lightView;

glProgramUniformMatrix4fv(_shadowProgram, loc, 1, GL_FALSE,
    glm::value_ptr(_lightViewProj));
_model.render(_shadowProgram);
```

2. **Geometry Pass (rendu final)** : dans le shader fragment, on transforme chaque fragment dans l'espace de la lumière, puis on compare sa profondeur avec celle stockée dans la shadow map. Le résultat (0 ou 1) permet de moduler la lumière diffuse et spéculaire.

Listing 3 – Fonction GLSL de comparaison

```
float computeShadow(vec3 fragPosWorld)
{
    vec4 lightSpacePos = uLightViewProj * vec4(fragPosWorld, 1.0);
    vec3 projCoords = lightSpacePos.xyz / lightSpacePos.w;
    projCoords = projCoords * 0.5 + 0.5;

    if (projCoords.x < 0.0 || projCoords.x > 1.0 ||
        projCoords.y < 0.0 || projCoords.y > 1.0)
        return 1.0;

    float closestDepth = texture(uShadowMap, projCoords.xy).r;
    float currentDepth = projCoords.z;
    return (currentDepth - uShadowBias > closestDepth) ? 0.3 : 1.0;
```

}

Un biais configurable a été ajouté pour éviter les artefacts d'acné dus à des imprécisions flottantes.

3.2.3 Justification des choix

Le shadow mapping est une méthode simple, rapide, et extensible pour générer des ombres dynamiques réalistes :

- **Efficace sur scènes complexes** : pas besoin d'analyser la topologie ou de connaître les intersections géométriques.
- **Indépendant de la géométrie** : les ombres sont définies en image (depth map), ce qui permet de gérer facilement des millions de triangles.
- **Facile à ajuster** : le biais, la taille de la zone orthographique, et la résolution de la texture peuvent être modifiés en temps réel.
- **Compatible avec pipeline différé** : on peut intégrer le test d'ombre dans le shader final sans multiplier les passes.

Le choix d'une lumière directionnelle avec projection orthographique est particulièrement bien adapté à une scène de type environnement extérieur comme ici.

3.3 Matériaux et textures

Support des maps :

- `diffuse`, `specular`, `normal`, `AO`
- Contrôles dynamiques via ImGui et uniformes conditionnels

3.4 Interface utilisateur

Interface ImGui permettant de :

- Contrôler les textures et matériaux
- Régler la position de la lumière
- Activer/désactiver FXAA et les ombres
- Ajuster le `bias` et la taille de projection orthographique

4 Limitations et pistes d'amélioration

- Ombres nettes : possibilité d'ajouter du **PCF** (Percentage Closer Filtering) pour des bords plus doux
- Support d'une seule source de lumière : possibilité d'étendre à plusieurs lumières ou à du Forward+
- Pas de visualisation temps réel de la depth map (UI désactivée)

5 Choix techniques

Le projet a été réalisé en s'appuyant sur l'architecture du TP6. Ce TP proposait un moteur de rendu de base intégrant une architecture modulaire, une gestion des matériaux via ImGui, et un pipeline différé.

À partir de cette base, j'ai :

- Intégré une passe de post-traitement pour le FXAA ;
- Ajouté une passe d'ombrage (shadow mapping) avec calcul de la matrice de projection de la lumière ;
- Étendu l'interface utilisateur pour contrôler dynamiquement les effets ;
- Affichage via fullscreen quad et VAO unique.

- Amélioré le rendu des matériaux en ajustant l'intensité spéculaire et le modèle d'éclairage (Phong/Blinn).

Cette approche permettait de conserver un pipeline cohérent tout en ajoutant des effets visuels avancés.

6 Problèmes rencontrés

Au cours du développement du projet, plusieurs problèmes techniques ont été rencontrés, nécessitant des phases de débogage approfondies. Voici les principaux obstacles et leur résolution :

Erreurs de compilation liées aux shaders

Un des problèmes fréquents a concerné des erreurs GLSL lors de la compilation des shaders. Un exemple concret est l'oubli de la variable `vUV` dans certains vertex shaders, alors qu'elle était attendue dans le fragment shader (pour les accès texture). Cela générerait des erreurs telles que :

```
error: 'vUV' : undeclared identifier
```

Ces erreurs sont souvent dues à une désynchronisation entre les deux shaders (vertex/fragment) ou à l'oubli de déclarer l'attribut dans le layout du shader d'entrée.

Résolution

- Vérification systématique des liaisons entre les variables d'entrée/sortie ('in' / 'out').
- Uniformisation des noms dans tous les shaders.
- Ajout d'un système de log pour capturer les erreurs GLSL au moment du 'glCompileShader'.

Erreurs OpenGL : GL_INVALID_OPERATION

Ce type d'erreur a été observé lors de l'exécution du programme. Il s'agissait principalement d'opérations interdites par OpenGL, comme le rendu sans VAO actif ou l'utilisation d'un shader non lié.

Exemples observés

- Appels à `glDrawArrays` sans avoir bindé de VAO.
- Tentatives d'activation de programmes OpenGL sans les avoir compilés/linkés correctement.

Résolution

- Mise en place d'un wrapper 'checkGLError()' pour localiser les appels fautifs.
- Ajout de vérifications après chaque chargement de shader et binding de VAO.
- Test unitaire des passes (geometry pass, shadow pass, post-process) indépendamment.

Conflits entre FXAA et les autres passes de rendu

Lors de l'activation simultanée de FXAA et d'autres effets comme le shadow mapping, des artefacts d'affichage (écran noir, scènes non visibles) ont pu apparaître. Cela provenait d'une mauvaise gestion des Framebuffers (FBOs) et du pipeline de post-traitement.

Origine du problème

- Le FXAA lisait dans une texture qui n'avait pas été écrite (pipeline mal ordonné).
- Certaines passes remplaçaient accidentellement le contenu du FBO principal.

Résolution

- Réorganisation stricte du pipeline : `shadowPass` → `geometryPass` → `fxaaPass`. - Isolation des FBOs pour chaque effet : aucun effet ne réutilise le même buffer sans le reconfigurer. - Ajout de drapeaux dans ImGui pour activer/désactiver proprement les passes.

Leçons tirées

Ces erreurs ont renforcé l'importance :

- de structurer proprement le pipeline de rendu multi-pass ;
- de maintenir une cohérence stricte dans les shaders (noms, types, layout) ;
- d'utiliser les outils de débogage OpenGL (comme `glGetError`) pour mieux diagnostiquer les problèmes de pipeline.

7 Comparaison

Effet du FXAA

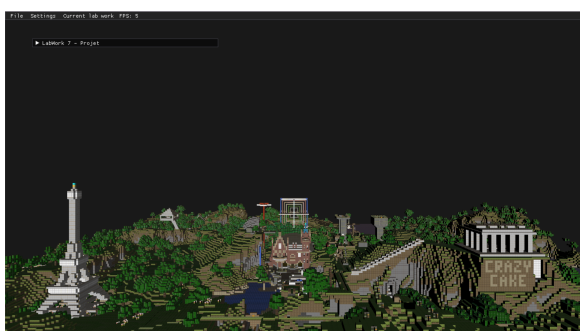
Sur la Figure 2, on observe que l'activation du FXAA permet de lisser efficacement les contours irréguliers, notamment sur les diagonales ou les objets lointains (ex. bâtiments, arbres). Sans FXAA, des effets de crénelage (aliasing) sont visibles. L'ajout de cet effet améliore donc significativement la lisibilité et le confort visuel sans altérer les détails de la scène.

Modèles d'éclairage : Phong vs Blinn-Phong

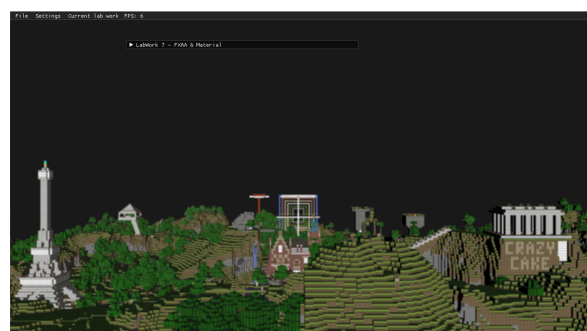
La Figure 3 illustre la différence entre les modèles Phong et Blinn-Phong. Le modèle Blinn-Phong produit des reflets spéculaires plus doux et centrés, car il utilise la demi-direction entre la lumière et la caméra. Phong, à l'inverse, peut générer des reflets plus localisés mais aussi plus "durs". Dans notre scène, Blinn-Phong s'avère plus agréable visuellement sur des surfaces comme les colonnes du temple.

Impact de l'intensité spéculaire

La Figure 4 met en évidence le rôle de l'intensité spéculaire. Lorsque cette valeur est proche de 1.0, les reflets sont très brillants, ce qui peut "brûler" les détails (saturation blanche). Une valeur plus faible donne un rendu plus subtil et réaliste. L'ajustement dynamique via l'interface ImGui est particulièrement utile pour trouver un bon équilibre visuel.



(a) FXAA désactivé



(b) FXAA activé

FIGURE 2 – Comparaison visuelle avec et sans FXAA

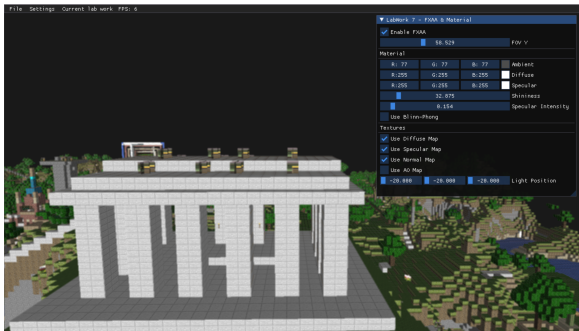


(a) Phong

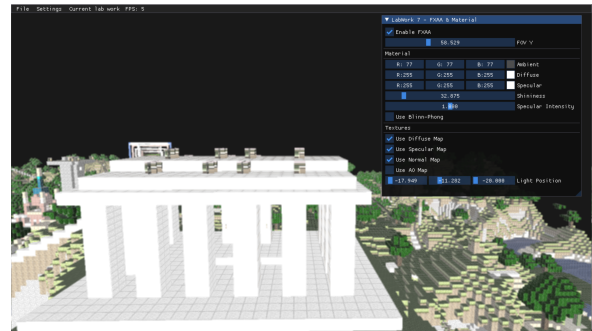


(b) Blinn-Phong

FIGURE 3 – Comparaison des modèles d'éclairage



(a) Specular inférieur à 1.0



(b) Specular égal à 1.0

FIGURE 4 – Effet de l'intensité spéculaire sur le rendu

8 Conclusion

Ce projet m'a permis d'approfondir mes connaissances des effets de post-traitement modernes en OpenGL. J'ai appris à structurer un pipeline de rendu avec plusieurs passes, intégrer des shaders complexes, manipuler des FBOs, et contrôler de manière interactive les paramètres de la scène.

Sources

- **LearnOpenGL – Shadow Mapping** :
<https://learnopengl.com/Advanced-Lighting/Shadows/Shadow-Mapping>
- **Khronos OpenGL Wiki – Shadow Mapping** :
https://www.khronos.org/opengl/wiki/Shadow_Mapping
- **Brendan Galea – Shadow Mapping Tutorial** :
<https://bgalea.com/tutorials/opengl/shadow-mapping>
- **Shader FXAA WebGL (GLSL)** :
<https://github.com/mitsuhiko/webgl-meincraft/blob/master/assets/shaders/fxaa.glsl>
- **FXAA – White Paper NVIDIA** :
https://developer.download.nvidia.com/assets/gamedev/files/sdk/11/FXAA_WhitePaper.pdf
- **LearnOpenGL – Anti-Aliasing (Post-Processing)** :
<https://learnopengl.com/Advanced-OpenGL/Anti-Aliasing>
- **Support pédagogique** :
 Documents et exemples fournis dans le cadre des TP M3D ISICG 2024/2025